

Silent failures: five ways a driver definition does nothing, and says nothing

A `#cmdSetup` control does not appear. A whole Setup dialog does not open. A command in `#initCmd` never reaches the instrument. A readback field shows a stale value that was correct ten minutes ago. In each case TestController logs no error, raises no dialog, and prints nothing in the Commands pane even with `debug` on. The definition looks right, the linter passes it, and the device is connected.

What this note collects, in five points:

1. A `#cmdSetup` control whose parameter line is missing, or does not split into the exact number of fields its type expects, prevents the **entire Setup dialog** from opening. Not just that control — the whole popup.
2. Under `#driver Ascii`, a `#scpiCmd` whose definition body is shorter than five characters does not dispatch. `tx S` is four characters and fails; `tx ("S")` is eight and works.
3. Under `#driver Ascii`, a name that does not resolve to a `#scpiCmd` is not passed through to the instrument. A raw instrument string in `#initCmd` goes nowhere.
4. `infoAsk` refreshes only from its own `Get` button. Naming one in another control's `:update:` list does nothing at all, and the field keeps displaying whatever the last `Get` returned. `info` does refresh.
5. A read that times out still runs `:readmath:`, which evaluates against no data and produces a plausible-looking answer. The field shows a value; nothing marks it as stale or absent.

The common thread is that all five are silent. None of them is a syntax error, so a linter cannot see them, and none produces a log line, so `debug` does not reveal them either. What you get is a definition that is quietly less functional than it reads.

Describes TestController 3.51. Written 5 September 2026. Details of this kind change between releases; anything here that a later version contradicts is out of date rather than disputed.

Authority. This is a working note, assembled from watching TestController behave while writing a driver for a non-SCPI instrument. It is not documentation, and it has not been reviewed by the author of TestController. HKJ is the author and the only authority on what the program is supposed to do; where this note and he disagree, he is right and this note is a bug report. Every claim below was reproduced against 3.51 on the bench, and the section for each one says exactly how.

1. A malformed parameter line takes the whole Setup dialog with it

Most `#cmdSetup` control types take a parameter line — the bare, untagged line or lines that follow the tags. `number` takes `unit min max`. `checkbox` takes `label offValue onValue`. `updater` takes an interval in seconds. These are not optional, and a control that does not get what it expects does not fail alone.

Reproduced. A definition with two `button` controls opened its Setup dialog normally. Adding one `checkbox` with tags but no parameter line, and changing nothing else:

```
#cmdSetup checkbox BadCheckbox Main
:write: LongCmd
:tip: deliberately missing its "label off on" params line
```

After a restart the device still connected — Found ZZ QuirkProbe on Dummy — and *Popups* → *Device* → *Show all setup popups* produced **no window at all**. Not an empty dialog, not a dialog missing one row: no window was created. The Commands pane logged nothing. Removing the checkbox restored the dialog.

This is worth knowing because the symptom points somewhere else entirely. A Setup dialog that will not open looks like a windowing problem, a focus problem, or a device that has not really connected, and all three are more likely explanations to reach for first. The actual cause is one control several lines further down.

What to check first. If a Setup dialog does not open, read every control's parameter line before investigating anything else:

Control	Parameter line
number, advNumber	unit min max
checkbox	label offValue onValue
combobox, buttons, radio	one label value line per entry
updater	interval in seconds

Note that `:string:` is a **tag**, not a parameter line. Using `:string: 0 off` in place of a checkbox's `Enabled 0 1` line leaves the control with nothing, and takes the dialog down.

2. A `#scpiCmd` body shorter than five characters does not dispatch

Under `#driver Ascii`, the definition body — everything after the command name — must be at least five characters long. Four-character bodies are silently dropped.

This bites the shortest and most obvious commands. An instrument whose standby command is the single letter `s` gives the natural definition `#scpiCmd Standby tx S`, and that body is `tx S`: four characters.

Reproduced. Two definitions differing only in body length:

```
#scpiCmd ShortCmd tx S
#scpiCmd LongCmd  tx ("S")
```

Both appear in the device's command list, so both parsed. Invoking each from the command line, with `debug on`:

```
LongCmd
ZZQP: Tx <LongCmd>
ZZQP: Tx <tx ("S")>

ShortCmd
ZZQP: Tx <ShortCmd>
```

A working dispatch logs two lines: the name, then the resolved definition. `ShortCmd` logs the name and stops. Nothing further is logged, no error appears, and nothing is transmitted.

The workaround is to make the body longer without changing what is sent. Wrapping the payload in a quoted expression does exactly that: `tx ("S")` is evaluated back to `S` before transmission, and the log above confirms the expression is what dispatches. Any command whose body would otherwise be four characters or fewer can be written this way.

3. Only names defined with `#scpiCmd` reach the instrument

The documentation for the `Ascii` driver says that "only commands defined with `#scpiCmd` is passed to the device". That is easy to read as a statement about the command line and to forget when writing `#initCmd`, `#finalCmd` or `#outputOff` — which take command *names*, not instrument strings.

Reproduced. A definition with `#initCmd RAWINIT`, where no `#scpiCmd` named `RAWINIT` exists, logs a single line at connect:

```
Dummy: Tx <RAWINIT>
```

and no resolved-definition line after it — the same signature as the failed dispatch in section 2, and unlike every successful dispatch, which logs the definition on a second line. Nothing is transmitted.

The correct form names a command and supplies its parameter. For an instrument whose delimiter command is `E1`:

```
#scpiCmd SetDelimiter tx E(value)
#initCmd SetDelimiter 1
```

which logs, on a real interface:

```
Tx <SetDelimiter 1>
Tx <tx E(value)>
Tx: <E1.> 45 31 0A
```

Three lines: the name, the definition it resolved to, and the bytes on the wire. That third line is the only proof the instrument heard anything, and it is missing whenever one of these failures occurs.

4. `infoAsk` does not refresh, `info` does

`:update:` names other controls to re-read after this control writes. Pointing it at an `infoAsk` does nothing whatsoever. The field is refreshed only when a person presses its own `Get` button, so it goes on displaying the result of the last press — which may be minutes old and describe a setting since changed.

Reproduced. A `number` control with `:update: Main.Status`, writing to an instrument whose status is displayed in an `infoAsk` named `Status`. Setting 5 V and pressing `Get` showed the correct value. Setting 7 V and touching nothing else left the field reading 5 V, while the same underlying query — issued for the number control's own `:read:` in the same instant — returned 7. A manual `Get` then showed 7.

This is very easy to misdiagnose as a timing race, and that is the reason this section exists. A readback that is correct when refreshed by hand and stale when refreshed by `:update:` looks exactly like a controller reading back faster than an instrument settles. On the instrument in question the settle time was measured at 14–40 ms, far too fast to explain it, and a deliberate update delay changed nothing — because no read was being issued at all.

Changing the control type from `infoAsk` to `info` fixed it with no other edit. `info` responds to `:update:` by name and re-reads. The cost is that `info` has no `Get` button, so it refreshes only when something else writes.

To refresh a display when nothing has written — after someone uses the front panel, say — add an `updater`, whose whole purpose is a timer that re-syncs the controls its own `:update:` names:

```
#cmdSetup updater Refresh Main
:update: Main.Status Main.Output_State_Readback
2
```

Two cautions on `updater`. Its interval line is subject to section 1: omit it and the Setup dialog will not open. And it polls the instrument for as long as the dialog is open, so against an unresponsive instrument it produces a continuous stream of timeouts that floods the Commands pane and makes the log useless for anything else.

5. A timed-out read still runs `:readmath:`

When a read times out, the `:readmath:` expression attached to it is still evaluated, against no data. If the expression cannot distinguish "no reply" from a real reply, it returns a perfectly ordinary-looking value, and the field displays it.

Observed, on a driver whose standby indicator tests for a `*` character in the status word:

```
Valhalla2701C: Tx <Status?>
Valhalla2701C: Rx Timeout
Valhalla2701C: Rx after :readmath: indexOf(value,"*") >= 0 ? "STANDBY" : "OPERATE" <OPERATE>
```

The instrument said nothing. The expression asked whether the nothing contained a `*`, correctly concluded that it did not, and the dialog displayed `OPERATE` — a definite statement about the instrument's output state, derived entirely from a failed read.

The lesson is not to avoid `:readmath:` but to write expressions whose false branch is the safe one, and, where it matters, to test for emptiness explicitly rather than letting it fall through to whichever branch happens to be last. A `:readmath:` is a statement about the reply; when there is no reply, it is a statement about nothing.

What these have in common

Four of the five are failures of *dispatch* rather than of *syntax*: the definition parses, the control or command exists, and nothing objects — the work simply is not done. The fifth inverts it, doing work on data that never arrived. In all five cases the Commands pane with `debug` is the tool that settles it, but only if you know what a *successful* exchange looks like, because these failures are visible as an absence:

```
Tx <CommandName>          the name was recognised
Tx <tx PAYLOAD(value)>    it resolved to a definition      <- missing on failure
Tx: <PAYLOAD.> 50 41 ...  the bytes reached the interface  <- missing on failure
```

One line means the dispatch died. Three lines mean the instrument heard you. Learning to read that shape is most of the debugging.

Sources

Documentation, on the author's site:

Page	Cited for
ConfigDevice	the three ascii drivers and their differences; <code>#cmdSetup</code> control types and their tags
ConfigDevice part 2	the <code>Ascii</code> driver, <code>#scpiCmd</code> , the <code>tx / txrx?</code> primitives, and "only commands defined with <code>#scpiCmd</code> is passed to the device"
Functions	<code>indexOf</code> , <code>match</code> , <code>getMatchGroup</code> and the other expression functions used in <code>:readmath:</code>

Everything else here is observation of the program's behaviour while writing a driver for it, reproduced against 3.51 on the bench, and is this note's own responsibility.

The author of TestController is the authority on all of this. Anything above that contradicts him is an error in this note.
